

ETIS

Installation & Administration Guide

Greenfield installation, production configuration, deployment, validation, administration, upgrade, and decommissioning

v0.18.1 candidate manual edition. Local UI validation only; production remains accepted v0.18.0 until protected deployment and human acceptance.

Audience	Deployment administrators, course owners, engineering maintainers, institutional IT and security staff
Purpose	Enable a new administrator to install ETIS Engineering Studio safely from source through accepted Azure production operation, without relying on undocumented tribal knowledge.
Reference implementation	COMP 330/474 Software Engineering • Fall 2026
Canonical service	https://simulator.etisframework.org
Version	v0.18.0 • October 2, 2026

CORE PRINCIPLE

AI may challenge, critique, synthesize, and ask for evidence. Engineers decide, defend, and own the consequences. Deterministic authorization, lifecycle state, and frozen repository evidence remain authoritative.

Contents

Executive Overview What this guide installs, who owns which boundary, and the canonical deployment path

- 1 Deployment model and installation sequence
- 2 Prerequisites and accounts
- 3 Clone, local installation, and verification
- 4 Microsoft Entra human authentication
- 5 GitHub OAuth identity linking
- 6 GitHub App repository evidence access
- 7 GitHub production environment, variables, and secrets
- 8 Azure infrastructure and resource topology
- 9 DNS, TLS, callbacks, and external alignment
- 10 First production deployment
- 11 Post-deployment acceptance
- 12 Routine administration and semester posture
- 13 Upgrade and rollback
- 14 Troubleshooting
- 15 Decommissioning

Appendix A Greenfield installation checklist

Appendix B Variables and secrets reference

Appendix C Acceptance and recovery commands

Executive Overview

ETIS Engineering Studio is a browser-based engineering apprenticeship environment. The production reference implementation uses Microsoft Entra for human authentication, database-backed course authorization, GitHub OAuth for engineering identity, a GitHub App for read-only repository evidence, Azure Container Apps for the web runtime, PostgreSQL Flexible Server for durable state, Key Vault and managed identity for secrets, Application Insights/Log Analytics for observability, ACR for immutable images, and OpenAI for bounded advisory review conversations.

THE INSTALLATION PRINCIPLE

Do not collapse independent trust boundaries. Entra authenticates the person. ETIS roster/staff records authorize course access. GitHub OAuth identifies the engineer's GitHub account. The GitHub App authorizes exact repository evidence access. Azure/Key Vault protect runtime infrastructure and secrets.

1. Prepare required administrative accounts and DNS authority.
2. Clone the repository and prove the source locally.
3. Register/configure Microsoft Entra authentication.
4. Configure GitHub OAuth identity linking.
5. Configure the ETIS Engineering Studio GitHub App for selected-repository read access.
6. Create the protected GitHub production environment and configure required variables/secrets.
7. Run the repository's controlled Deploy Azure workflow; the workflow provisions/updates Azure from Bicep, builds an immutable image, runs migrations, deploys the application, and verifies readiness.
8. Bind DNS/TLS and verify callbacks.
9. Run the Azure Operations CLI acceptance report and complete human identity/GitHub onboarding acceptance.
10. Only after acceptance, activate the semester for normal student use.

Controlled production delivery path

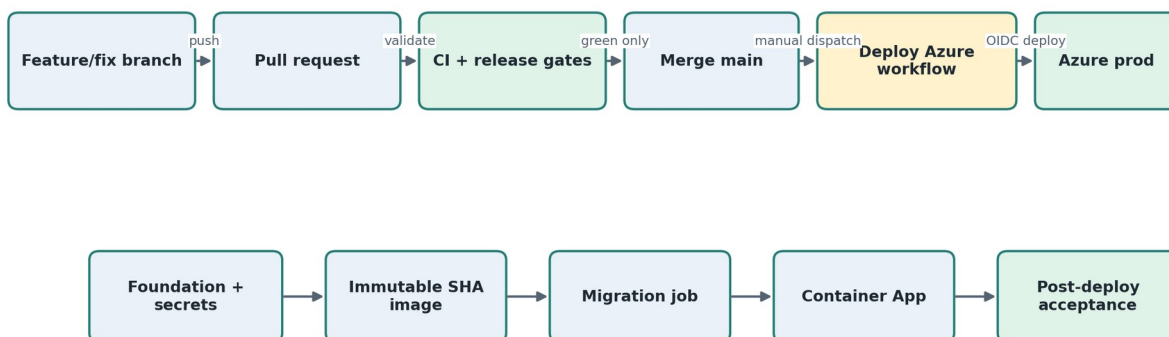


Figure 1. Canonical production delivery path. Deployment remains branch → PR → CI → merge → protected Deploy Azure workflow → post-deployment acceptance. Historical reference figure; current text governs.

1. Deployment model and installation sequence

1.1 What is automated and what is not

Area	Administrator responsibility	Repository/workflow responsibility
External identity	Create/configure Entra application and GitHub integrations; protect credentials.	Validate required configuration and fail closed if production values are missing or unsafe.
Azure infrastructure	Provide subscription authority and protected deployment credentials.	Bicep defines networking, ACR, Key Vault, managed identity, Container Apps environment/app/job, PostgreSQL, monitoring, alerts.
Database schema	Protect PostgreSQL credentials and recovery posture.	Alembic migrations are explicit release artifacts and run before app deployment.
DNS/TLS	Control public DNS and ensure canonical origin/callback alignment.	Application requires HTTPS production origin and exposes health/readiness endpoints.
Semester/course data	Create term/sections, import roster, assign staff/teams, activate/archive.	Server enforces lifecycle-valid authorization and preserves historical records.

SOURCE OF TRUTH

Repository infrastructure and the protected GitHub workflow are authoritative for normal production deployment. Avoid one-off portal changes that are not represented in Bicep; they can create drift and may be overwritten by a later deployment.

2. Prerequisites and accounts

Area	Required before installation
Source control	GitHub repository access, ability to create PRs, manage repository environments/secrets, and manually run the protected production workflow.
Local workstation	Git, Python 3, a Python virtual environment, browser, curl, and Azure CLI for production operations. Node is not required for the bundled UI.
Azure	Subscription with authority to deploy Container Apps, PostgreSQL Flexible Server, ACR, Key Vault, managed identity, networking/private DNS, Log Analytics, Application Insights, Monitor alerts/action groups.

E ETIS ENGINEERING STUDIO INSTALLATION & ADMINISTRATION GUIDE	
Area	Required before installation
Microsoft Entra	Ability to create/configure the production app registration and client secret in the intended tenant.
GitHub integrations	Ability to create/configure the GitHub OAuth App and GitHub App.
OpenAI	API project/key with the configured reviewer model available.
DNS	Control of the intended public DNS name. Reference implementation: simulator.etisframework.org .
Operations	An email destination for Azure operational alerts and an owner for budget/incident response.

3. Clone, local installation, and verification

3.1 Clone and create the Python environment

```
git clone https://github.com/etis-framework/etis-engineering-studio.git
cd etis-engineering-studio
cp .env.example .env
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-dev.txt
```

LOCAL ENVIRONMENT FILE

Use `.env` only for local development. Keep it excluded from Git and Docker build contexts. Never paste production secrets into `.env.example`, documentation, screenshots, test fixtures, or source control.

3.2 Run the test suite before local startup

```
python -m pytest -q
```

Expected result: the repository test suite passes. Do not proceed to a production deployment from a source tree with failing release/security tests.

3.3 Start the local application

```
uvicorn apps.api.app.main:app --reload --port 8000
```

Open <http://localhost:8000>. Development behavior may use local/demo paths when external providers are not configured. Production validation is intentionally stricter and rejects development login, SQLite, weak session secrets, missing provider configuration, and non-HTTPS production origins.

3.4 Optional local PostgreSQL path

```
docker compose up --build
```

Use the containerized PostgreSQL path when validating database-specific behavior locally. PostgreSQL concurrency and migration proofs also run in controlled CI/release gates.

Release installation and validation

The v0.18.0 source version is defined in `apps/api/app/version.py`. FastAPI/OpenAPI and `/health` use that value. Source version, merged Git SHA, immutable image/revision, deployment success and production acceptance are separate evidence. The August GO records and v0.17 analytical history keep their original dates and versions.

Accepted production uses shadow/shadow evaluation with legacy student-visible authority. Shadow validator/planner/selector/realizer evaluation remains internal and does not replace student-visible authority. This release introduces no PR5 authority transfer, database migration or course-model change.

Install development/test dependencies with `python -m pip install -r requirements-dev.txt`. The development pytest pin is 9.0.3; production PyJWT is pinned to 2.15.1. Review open advisories against the current manifest graph rather than assuming a local install clears GitHub alerts.

Run `python -m pytest -q`, `node tests/review_journey_wargame.cjs` and `node tests/dialog_keyboard_wargame.cjs`. The optional real-browser keyboard runner requires Playwright and Chromium; Bicep, PostgreSQL and production-container proofs run in CI when local prerequisites are absent.

4. Microsoft Entra human authentication

Create or use a production Microsoft Entra application registration for ETIS Engineering Studio. Human authentication is separate from course authorization; a valid Microsoft identity does not grant course access unless ETIS has a current roster/staff assignment.

Setting	Production rule
Tenant	Use <code>ENTRA_AUTHORITY=organizations</code> for Microsoft authentication; Studio separately verifies <code>ENTRA_ALLOWED_TENANT_ID</code> , domain and immutable (tid, oid). Authentication authority does not grant admission.
Redirect URI	<code>https://simulator.etisframework.org/auth/entra/callback</code>
Protocol	OAuth 2.0 / OpenID Connect authorization code
Scopes	<code>openid profile email</code>
Client ID	Store as protected production environment configuration.
Client secret	Store only as a protected production secret; rotate if exposed.
MFA	Do not create a second ETIS-specific MFA scheme. Normal Loyola users satisfy their normal institutional MFA, then return to Studio.

AUTHENTICATION IS NOT AUTHORIZATION

A user who can complete Entra sign-in but is not in an active course/section/staff assignment should still be denied course operation. Treat “can sign in” and “can enter the course” as separate acceptance checks.

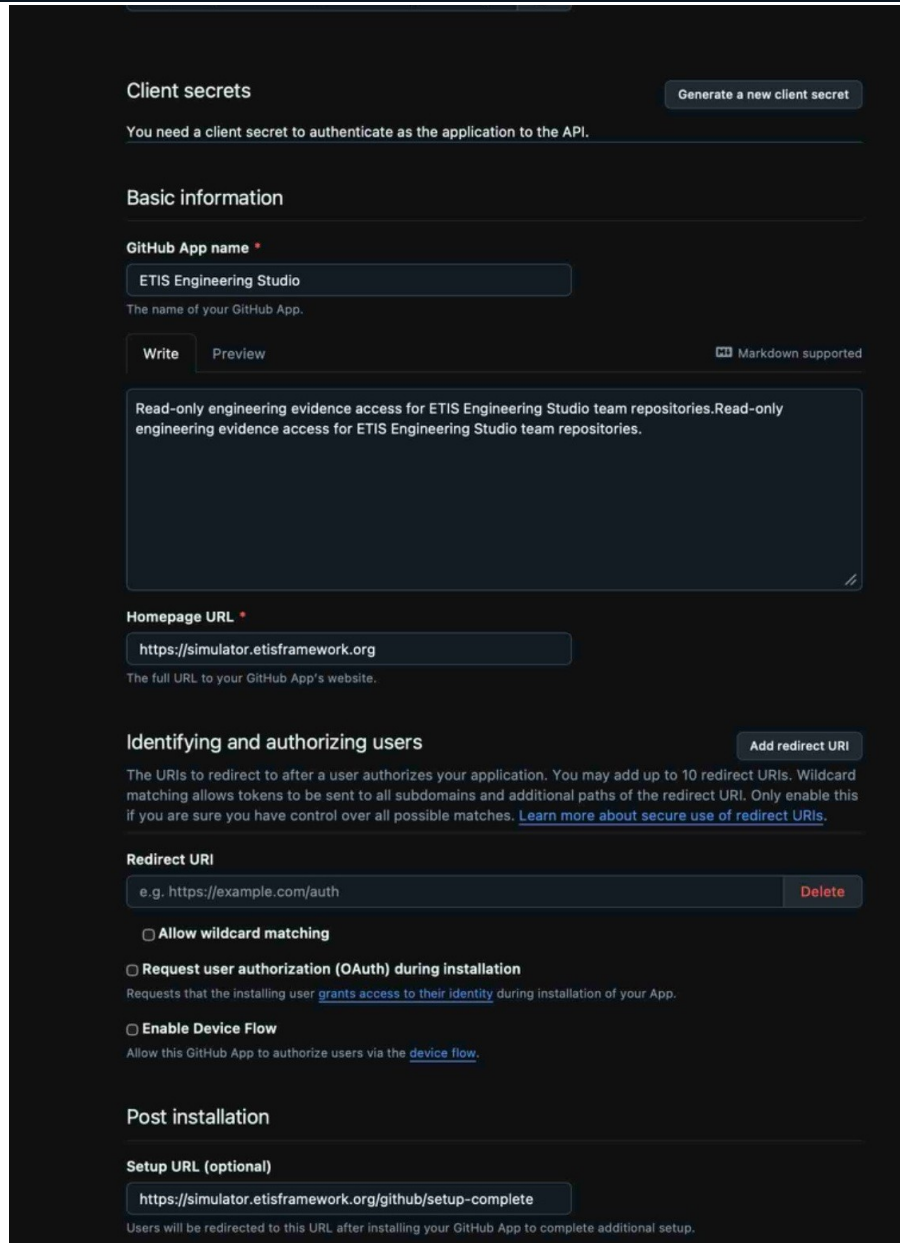
5. GitHub OAuth identity linking

The GitHub OAuth App links a Studio user to a GitHub login and immutable numeric GitHub account ID. It is an identity boundary only; repository evidence access is handled separately by the GitHub App.

Setting	Rule
Callback	https://simulator.etisframework.org/auth/github/callback
Scope	No repository scope is required for identity linking; ETIS intentionally avoids using OAuth repository tokens.
Account selection	Request explicit account selection so a student with multiple GitHub sessions can choose the intended engineering identity.
Persistence	Retain the GitHub login and immutable account ID; do not retain the OAuth access token for repository reading.

6. GitHub App repository evidence access

The ETIS Engineering Studio GitHub App is the repository trust boundary. It must support both personal and organization-owned repositories while remaining read-only and narrowly scoped.



Client secrets [Generate a new client secret](#)

You need a client secret to authenticate as the application to the API.

Basic information

GitHub App name *

ETIS Engineering Studio

The name of your GitHub App.

Write **Preview** [Markdown supported](#)

Read-only engineering evidence access for ETIS Engineering Studio team repositories. Read-only engineering evidence access for ETIS Engineering Studio team repositories.

Homepage URL *

https://simulator.etisframework.org

The full URL to your GitHub App's website.

Identifying and authorizing users [Add redirect URI](#)

The URIs to redirect to after a user authorizes your application. You may add up to 10 redirect URIs. Wildcard matching allows tokens to be sent to all subdomains and additional paths of the redirect URI. Only enable this if you are sure you have control over all possible matches. [Learn more about secure use of redirect URIs.](#)

Redirect URI

e.g. https://example.com/auth [Delete](#)

☐ **Allow wildcard matching**

☐ **Request user authorization (OAuth) during installation**

Requests that the installing user [grants access to their identity](#) during installation of your App.

☐ **Enable Device Flow**

Allow this GitHub App to authorize users via the [device flow](#).

Post installation

Setup URL (optional)

https://simulator.etisframework.org/github/setup-complete

Users will be redirected to this URL after installing your GitHub App to complete additional setup.

Figure 2. GitHub App general configuration. The production Setup URL returns the browser to ETIS after installation/update, and Redirect on update is enabled. Historical reference figure; current text governs.

GitHub App setting	Accepted configuration
Name	ETIS Engineering Studio
Visibility/installability	Public, so personal GitHub accounts can install it; this does not make repositories public.
Homepage	https://simulator.etisframework.org
Setup URL	https://simulator.etisframework.org/github/setup-complete

E ETIS ENGINEERING STUDIO INSTALLATION & ADMINISTRATION GUIDE	
GitHub App setting	Accepted configuration
Redirect on update	Enabled
Webhooks	Disabled in the current implementation; there is no webhook receiver.
Organization permissions	None required
Repository selection	Only select repositories. ETIS rejects repository_selection=all.

6.1 Repository permissions

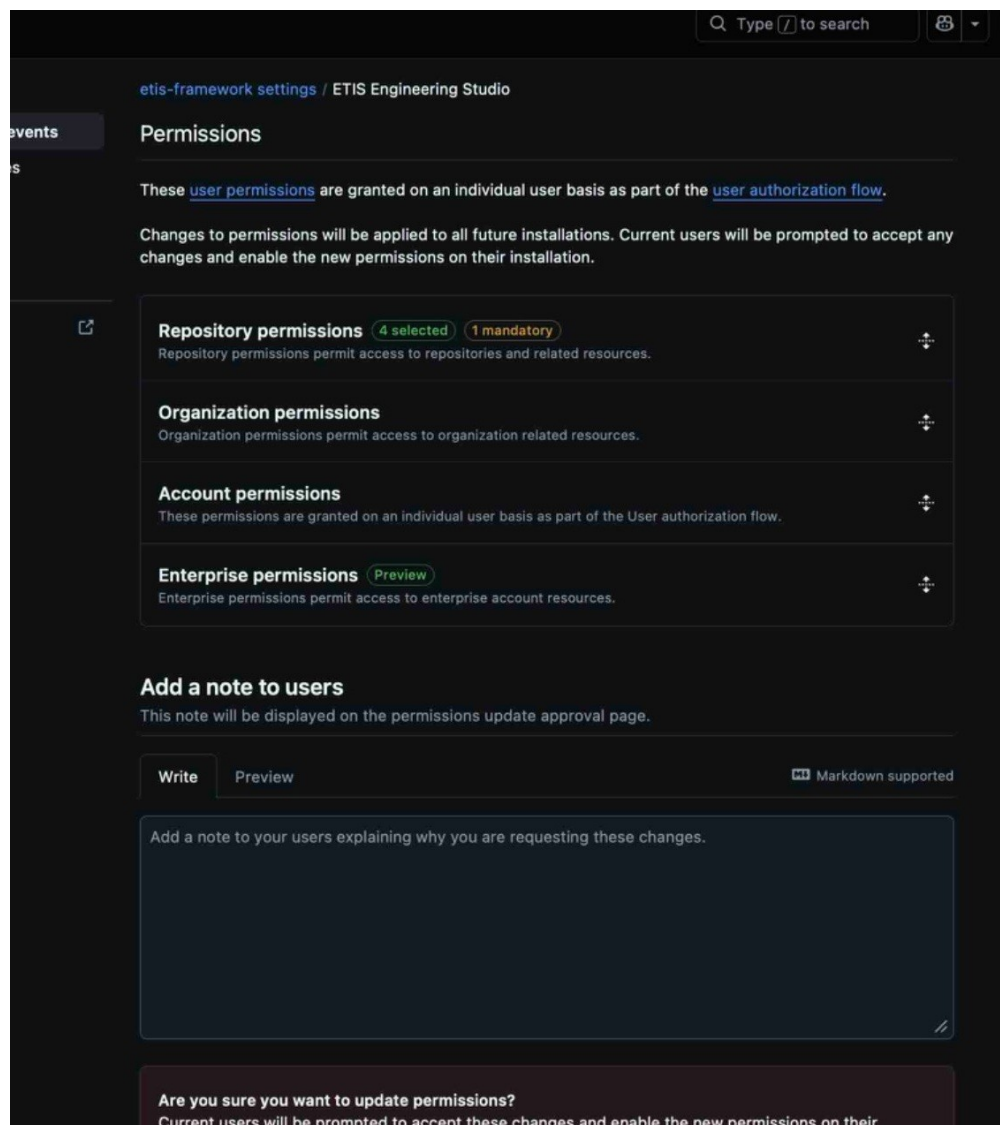


Figure 3. GitHub App permissions surface. Keep repository access read-only and limited to the permissions required for evidence acquisition. Historical reference figure; current text governs.

Permission	Access
Metadata	Read
Contents	Read
Issues	Read
Pull requests	Read
Actions	Read

SELECTED REPOSITORIES ONLY

Every installation used by ETIS must choose “Only select repositories.” An installation that grants All repositories fails closed. Installation tokens are requested for the exact nominated repository only.

6.2 Private key handling

- Generate the GitHub App private key using GitHub's App administration surface.
- Store the private key only in the protected production secret path used by the deployment workflow.
- Never commit the .pem key, paste it into documentation, or retain copies in shared course folders.
- Rotate/revoke the key immediately if exposure is suspected.

7. GitHub production environment, variables, and secrets

The repository uses a protected GitHub environment named production. The deployment workflow acquires Azure authority through OIDC only after source/release validation. Normal production deployment should be performed from main through the manual Deploy Azure workflow.

The screenshot displays the 'Configure production' environment settings in GitHub Actions. The interface is dark-themed with a sidebar on the left containing navigation links: 'Agents', 'Actions', 'Projects', 'Security and quality', 'Insights', and 'Settings'. The main content area is titled 'Environments / Configure production'.

Deployment protection rules

Configure reviewers, timers, and custom rules that must pass before deployments to this environment can proceed.

- ☐ **Required reviewers**
Specify people or teams that may approve workflow runs when they access this environment.
- ☐ **Wait timer**
Set an amount of time to wait before allowing deployments to proceed.
- ☐ **Enable custom rules with GitHub Apps** [Preview](#)
[Learn about existing apps](#) or [create your own protection rules](#) so you can deploy with confidence.
- ☒ **Allow administrators to bypass configured protection rules**

[Save protection rules](#)

Deployment branches and tags

Limit which branches and tags can deploy to this environment based on rules or naming patterns.

[Selected branches and tags](#)

1 branch and 0 tags allowed

`main` Currently applies to 1 branch [Edit](#) [Remove](#)

[+ Add deployment branch or tag rule](#)

Environment secrets [Add environment secret](#)

Secrets are encrypted environment variables. They are accessible only by GitHub Actions in the context of this environment by using the [secret context](#).

Name	Last updated
<code>AZURE_CLIENT_ID</code>	5 days ago
<code>AZURE_SUBSCRIPTION_ID</code>	5 days ago
<code>AZURE_TENANT_ID</code>	5 days ago
<code>ENTRA_CLIENT_SECRET</code>	5 days ago
<code>ETIS_GITHUB_APP_PRIVATE_KEY</code>	5 days ago
<code>ETIS_GITHUB_OAUTH_CLIENT_SECRET</code>	5 days ago
<code>ETIS_SESSION_SECRET</code>	5 days ago
<code>OPENAI_API_KEY</code>	5 days ago
<code>POSTGRES_ADMIN_PASSWORD</code>	5 days ago

Environment variables [Add environment variable](#)

Variables are used for non-sensitive configuration data. They are accessible only by GitHub Actions in the context of this environment by using the [variable context](#).

Name	Value	Last updated
------	-------	--------------

Figure 4. GitHub production environment (upper portion): deployment protection and branch restrictions plus the protected secret-name inventory. Secret values are not displayed. Historical reference figure; current text governs.

[secret context.](#)

Name ↕	Last updated
AZURE_CLIENT_ID	5 days ago
AZURE_SUBSCRIPTION_ID	5 days ago
AZURE_TENANT_ID	5 days ago
ENTRA_CLIENT_SECRET	5 days ago
ETIS_GITHUB_APP_PRIVATE_KEY	5 days ago
ETIS_GITHUB_OAUTH_CLIENT_SECRET	5 days ago
ETIS_SESSION_SECRET	5 days ago
OPENAI_API_KEY	5 days ago
POSTGRES_ADMIN_PASSWORD	5 days ago

Environment variables Add environment variable

Variables are used for non-sensitive configuration data. They are accessible only by GitHub Actions in the context of this environment by using the [variable context](#).

Name ↕	Value	Last updated
AZURE_LOCATION	centralus	5 days ago
AZURE_RESOURCE_GROUP	etis-studio-prod	5 days ago
ENTRA_CLIENT_ID	ee826386-507d-49a2-9794-c236dace6d7e	5 days ago
ETIS_BOOTSTRAP_OWNER_EMAIL	woconnell@luc.edu	4 days ago
ETIS_CUSTOM_DOMAIN_NAME	simulator.etisframework.org	4 days ago
ETIS_GITHUB_APP_ID	4641755	5 days ago
ETIS_GITHUB_APP_SLUG	etis-engineering-studio	5 days ago
ETIS_GITHUB_OAUTH_CLIENT_ID	Ov23lIKwX17bTJxQSwIV	5 days ago
ETIS_MANAGED_CERTIFICATE_NAME	mc-etis-studio-pr-simulator-etisfr-2126	4 days ago
ETIS_PRODUCTION_TEST_SECTION_KEY	PRODUCTION-TEST	5 days ago
ETIS_PRODUCTION_TEST_STUDENT_EMAIL	usranger290@gmail.com	5 days ago
ETIS_PRODUCTION_TEST_STUDENT_ID	production-test-student	5 days ago
ETIS_PRODUCTION_TEST_STUDENT_OID	a97f7cbb-f60f-4995-afb9-a49220b5ea09	5 days ago
ETIS_PRODUCTION_TEST_TEAM_KEY	production-test-team	5 days ago
ETIS_WEB_ORIGIN	https://simulator.etisframework.org	5 days ago
OPERATIONS_ALERT_EMAIL	woconnell@luc.edu	5 days ago

Figure 5. GitHub production environment (lower portion): continuation of secret names and the non-secret production variable inventory. Historical reference figure; current text governs.

PRODUCTION ENVIRONMENT BOUNDARY

Restrict deployment branches/tags to the intended release source (accepted baseline: main). Treat changes to environment protection, secrets, or variables as production configuration changes requiring review.

7.1 Protected secrets

Secret name	Purpose
AZURE_CLIENT_ID	OIDC deployment application/client identity
AZURE_TENANT_ID	Azure/Entra tenant identifier used by deployment
AZURE_SUBSCRIPTION_ID	Target Azure subscription
POSTGRES_ADMIN_PASSWORD	PostgreSQL administrative bootstrap/secret provisioning
ETIS_SESSION_SECRET	Application session signing/entropy secret
ENTRA_CLIENT_SECRET	Entra application secret
ETIS_GITHUB_APP_PRIVATE_KEY	GitHub App private key
ETIS_GITHUB_OAUTH_CLIENT_SECRET	GitHub OAuth App client secret
OPENAI_API_KEY	OpenAI API project key

7.2 Non-secret production variables

Variable name	Purpose / expected relationship
AZURE_RESOURCE_GROUP	Production resource group; reference implementation etis-studio-prod
AZURE_LOCATION	Azure region
ETIS_WEB_ORIGIN	Canonical HTTPS public origin
ETIS_BOOTSTRAP_OWNER_EMAIL	Initial Course Owner bootstrap identity
ETIS_CUSTOM_DOMAIN_NAME	Public custom domain
ETIS_MANAGED_CERTIFICATE_NAME	Managed certificate resource name
ENTRA_CLIENT_ID	Entra application client ID
ETIS_GITHUB_APP_ID	GitHub App numeric ID
ETIS_GITHUB_APP_SLUG	GitHub App slug
ETIS_GITHUB_OAUTH_CLIENT_ID	GitHub OAuth identity-linking client ID

Variable name	Purpose / expected relationship
OPERATIONS_ALERT_EMAIL	Operational alert destination
ETIS_PRODUCTION_TEST_*	Exact production acceptance test-student identity/section/team values; never use these as a broad external-domain allow list.

8. Azure infrastructure and resource topology

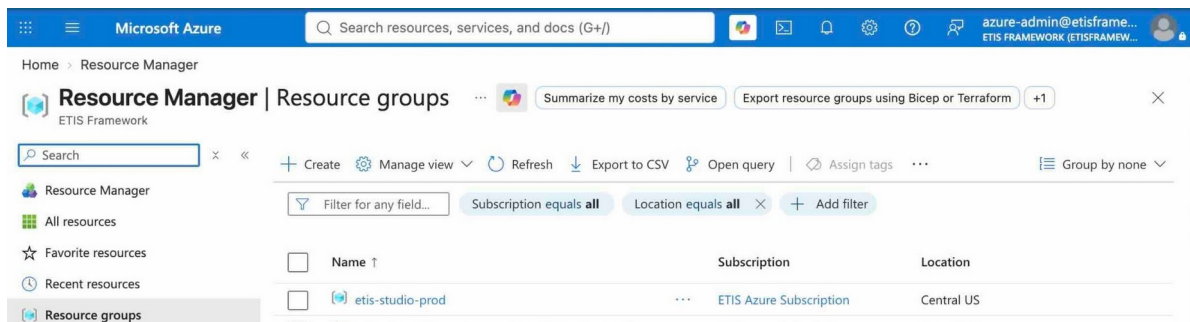


Figure 6. Azure resource-group view. The reference deployment keeps ETIS production resources under the dedicated *etis-studio-prod* resource group, plus platform-managed supporting groups where Azure requires them. Historical reference figure; current text governs.

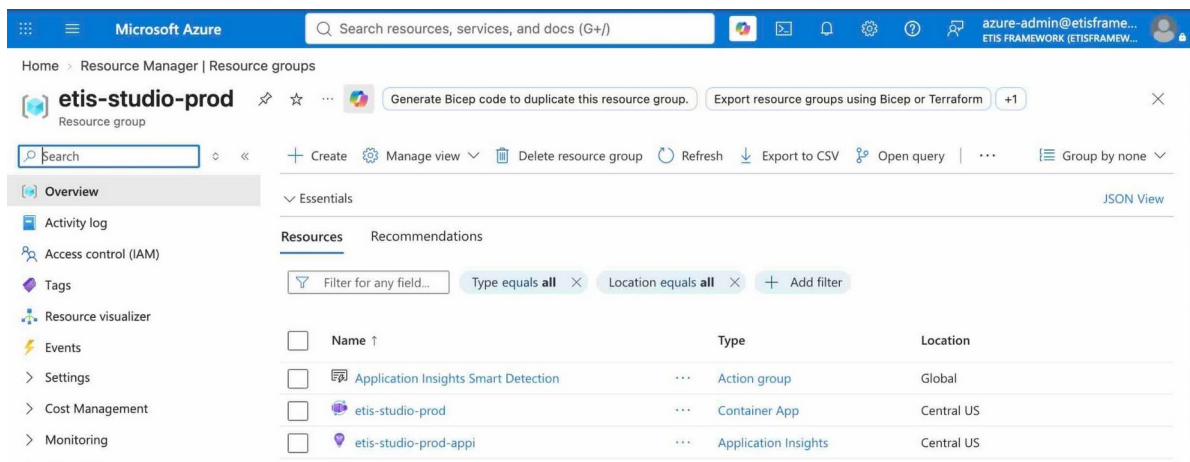


Figure 7. Production resource-group inventory. Key resources include Container Apps, Application Insights, Container Apps Environment, Key Vault, Log Analytics, migration job, alerts/action groups, PostgreSQL, ACR, managed identity, private networking, and supporting resources. Historical reference figure; current text governs.

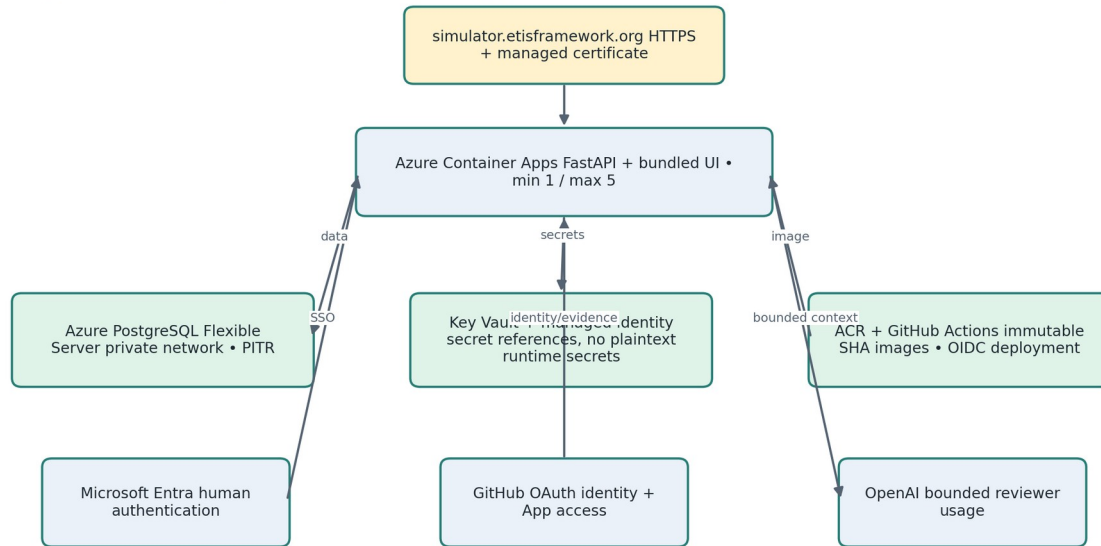
Accepted Azure production topology

Figure 8. Accepted production topology. External providers remain separate trust boundaries around the managed Azure runtime. Historical reference figure; current text governs.

8.1 Container Apps runtime

- Public HTTPS application endpoint through Azure Container Apps.
- Single revision mode in the accepted production baseline.
- Application container uses a user-assigned managed identity for ACR pull and Key Vault secret access.
- Runtime configuration references Key Vault secrets; do not place literal secrets in normal environment variables.
- Liveness/readiness probes are configured; /ready must fail closed on database connectivity or migration mismatch.

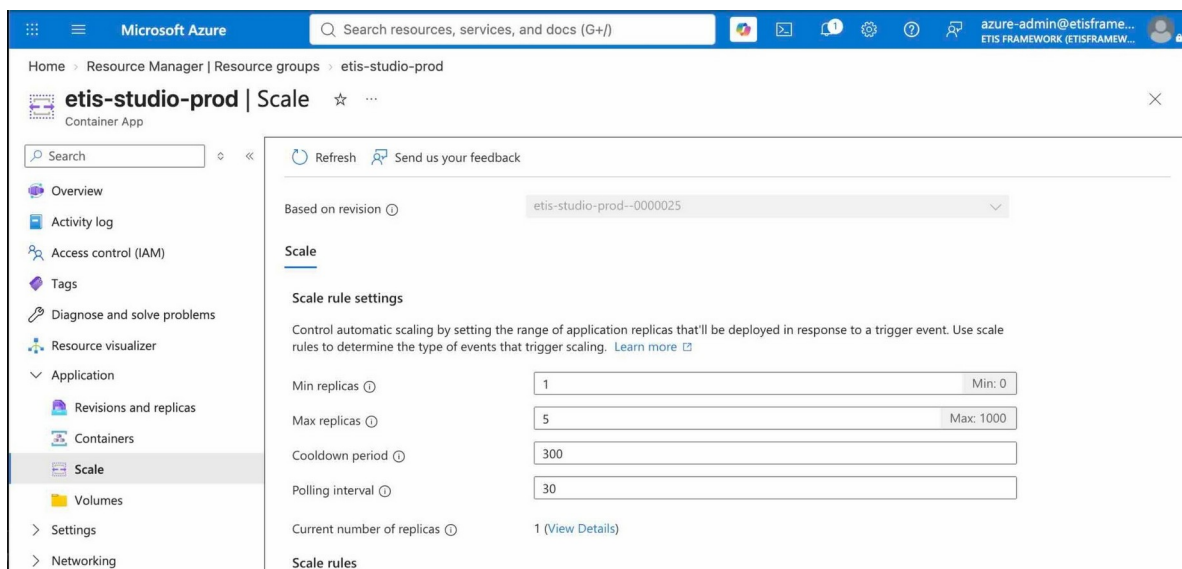


Figure 9. Semester production scaling baseline: minimum 1 replica, maximum 5 replicas. Keeping one replica warm avoids scale-to-zero cold starts during normal course operation. Historical reference figure; current text governs.

IAC IS AUTHORITATIVE

The accepted source baseline now sets minReplicas=1 and maxReplicas=5 in the application Bicep. Do not preserve an old note that Bicep defaults to 0; that was corrected before this publication.

8.2 PostgreSQL Flexible Server

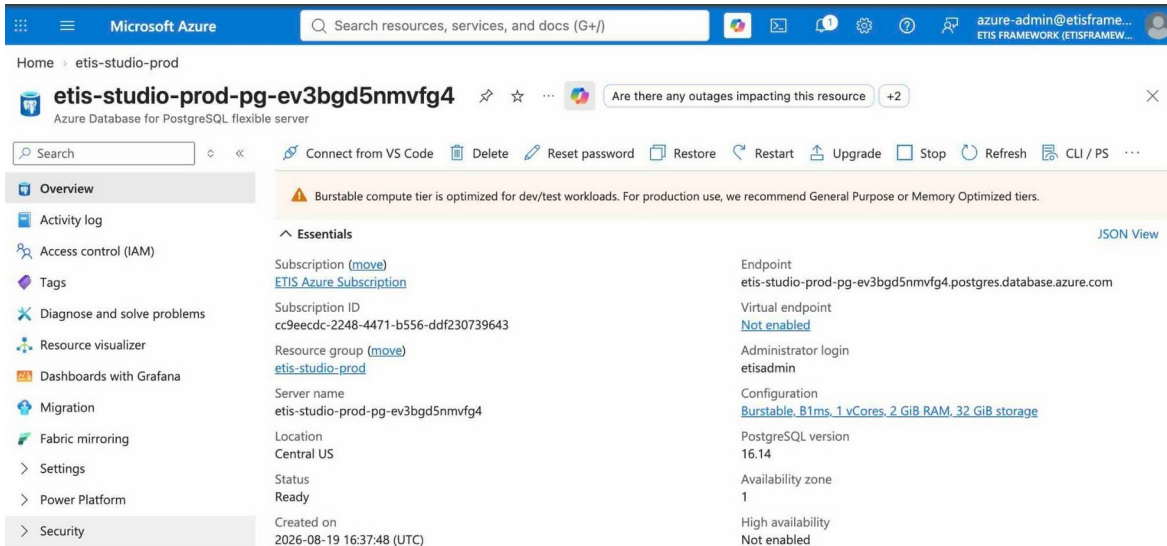


Figure 10. PostgreSQL Flexible Server overview. Production uses PostgreSQL as the durable system of record; schema lifecycle is controlled by Alembic. Historical reference figure; current text governs.

- Production baseline uses PostgreSQL 16 with private/VNet-integrated connectivity.
- Backup retention/PITR must be enabled and validated before broad student use.
- The reference environment uses a cost-conscious small-course compute posture; capacity should be reviewed if course load or institutional use grows.
- Application readiness checks require database connectivity and migration_current=true.

8.3 Key Vault and managed identity

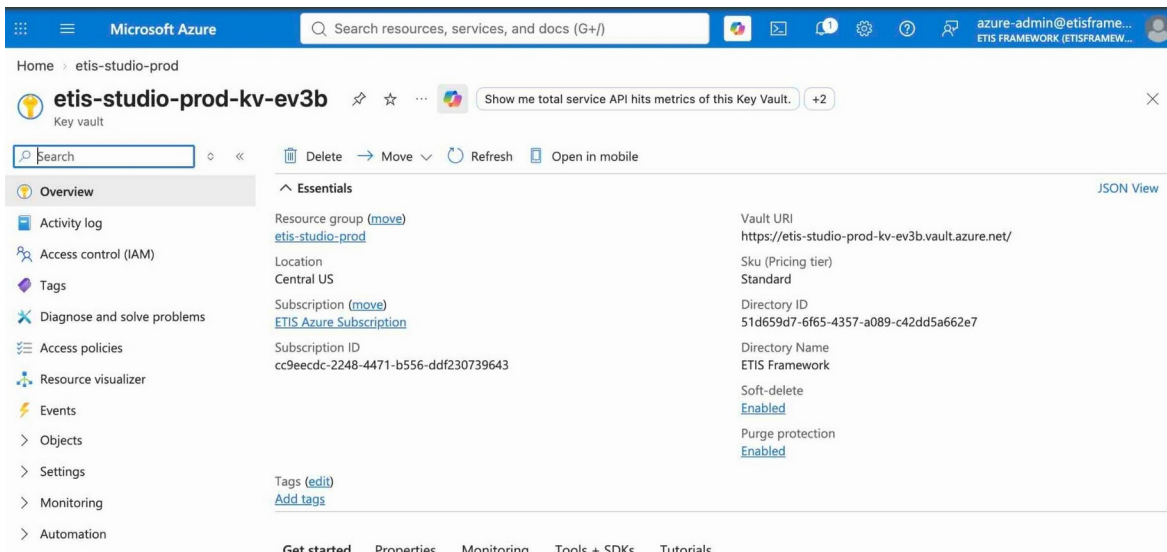


Figure 11. Key Vault overview. Document the vault and access model, never secret values. Historical reference figure; current text governs.

- Use Azure Key Vault with RBAC authorization.
- Grant the runtime managed identity Key Vault Secrets User at the production vault scope; do not grant application runtime broad vault administration.
- ACR administrative credentials remain disabled; use managed-identity AcrPull.
- Secret names may be documented; secret values must not be.

9. DNS, TLS, callbacks, and external alignment

Item	Production value / requirement
Canonical origin	https://simulator.etisframework.org
DNS	CNAME/custom-domain configuration must point the public host at the Container App target used by the deployment.
TLS	Managed certificate, HTTPS only, expected HSTS/security headers.
Entra callback	https://simulator.etisframework.org/auth/entra/callback
GitHub OAuth callback	https://simulator.etisframework.org/auth/github/callback
GitHub App Setup URL	https://simulator.etisframework.org/github/setup-complete
PostgreSQL	Private network/DNS path only; no public database exposure in the accepted design.

CALLBACKS MUST MATCH EXACTLY

Authentication failures after a technically successful Azure deployment are often alignment failures among ETIS_WEB_ORIGIN, DNS/TLS, Entra callback, GitHub OAuth callback, and GitHub App Setup URL. Verify them as one set.

10. First production deployment

10.1 Pre-deployment source gate

1. Create a feature/fix branch from current main.
2. Run focused tests and the full pytest suite.
3. Open a PR and require green CI/release gates.
4. Merge to main only after review and green CI.
5. Do not attempt to bypass protected-main or the production environment.

10.2 Run the protected Deploy Azure workflow

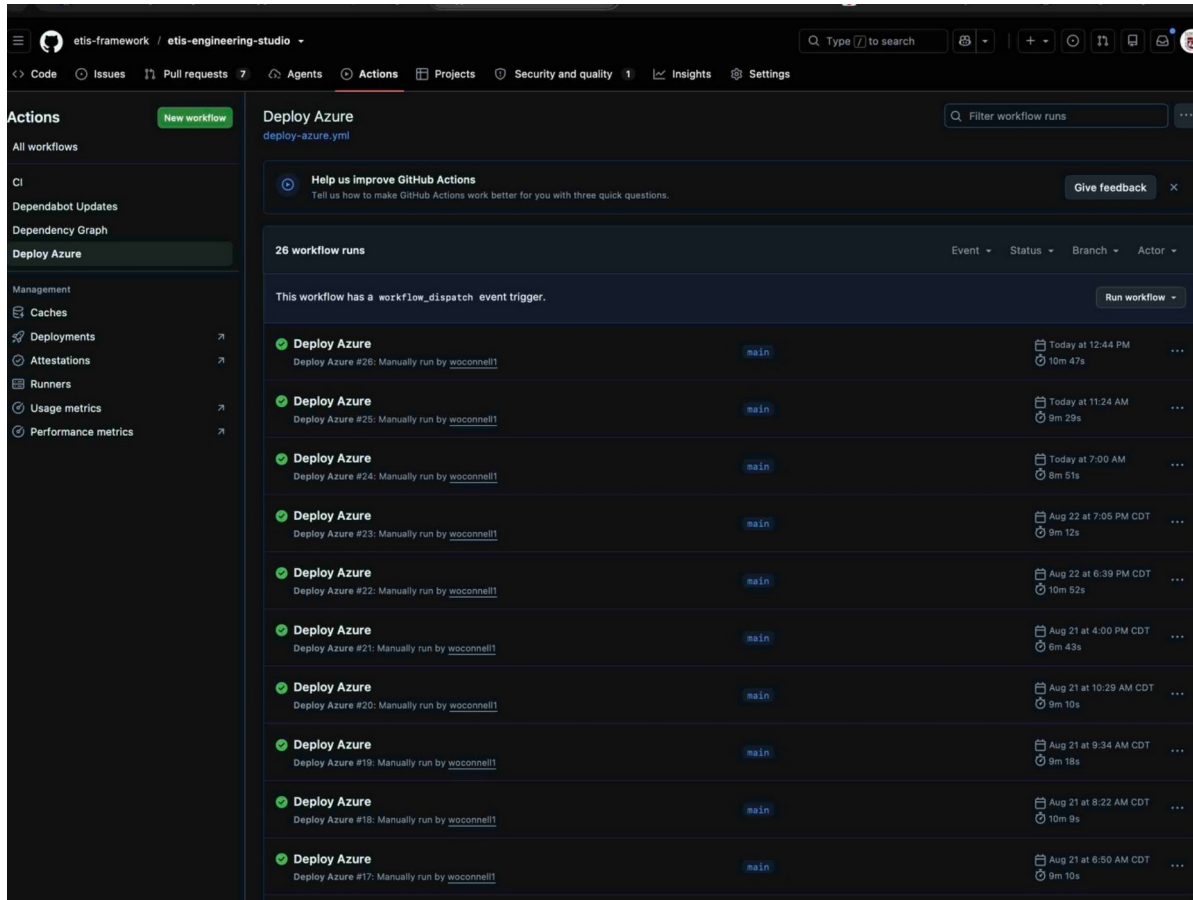


Figure 12. Deploy Azure workflow history. Production deployment is manually dispatched after merge to main. Historical reference figure; current text governs.

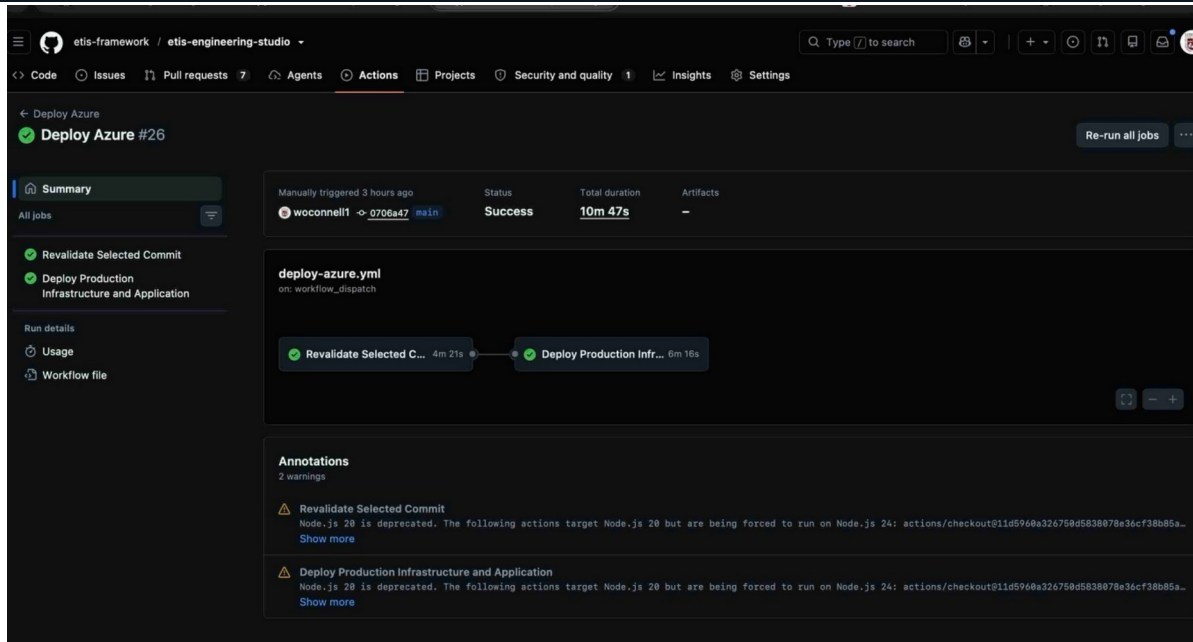


Figure 13. Successful deployment summary. The workflow first revalidates the selected commit, then deploys production infrastructure and the application. Historical reference figure; current text governs.

The workflow's controlled order is: release validation → Azure login (OIDC) → foundation → Key Vault secrets → immutable image → migration job → application → operational controls → readiness verification.

DO NOT BYPASS THE WORKFLOW

A green source build is not production authorization. Use the protected production environment and manual Deploy Azure workflow so the deployed image is tied to the validated Git SHA and production configuration.

11. Post-deployment acceptance

After every production deployment, use the repository's Azure Operations CLI for repeatable acceptance rather than relying on portal screenshots or one-off raw Azure commands.

```
./scripts/azure/etis-azure doctor
./scripts/azure/etis-azure status
./scripts/azure/etis-azure acceptance
```

A healthy acceptance report should show required checks PASS for production status, application health, replica capacity, external smoke test, runtime drift, and budget controls. Cost telemetry is informational and may WARN if Azure Cost Management is throttled; it must not silently become a production-health gate.

Acceptance area	What must be true
Health	/health returns 200 and production status is healthy.
Readiness	/ready returns 200, database_connected=true, migration_current=true, and application can serve normal traffic.

Acceptance area	What must be true
Replica posture	Runtime matches min 1 / max 5 baseline with a ready running replica.
Authentication	Loyola user returns from normal institutional sign-in/MFA without a second ETIS password/MFA enrollment.
GitHub identity	Link/relink returns to Studio and remains bound to the initiating session.
Repository onboarding	Personal and organization repositories reach verified state through Step 1 authorization plus Step 2 exact verification.
Secrets	Runtime uses managed identity/Key Vault references; no plaintext secret drift.
Observability	Application Insights/Log Analytics connected; alerts/action group active.
Recovery	PITR window exists and restore procedure has been validated before broad use.
Cost controls	\$100 monthly budget and 50/80/100% actual-cost alerts are configured.

Production cost governance

Accepted monthly budget: \$100 • actual-cost notifications • advisory, not an automatic shutdown



Operator response: confirm scope → investigate usage → compare trend → act only if operationally justified

Figure 14. Accepted cost-control posture. Budget alerts notify operators; they do not automatically stop or scale down production. Historical reference figure; current text governs.

Current-release production smoke

The Review Room is the conversation home. Engineering Evidence is the saved source notebook. Same-snapshot and newer-saved-snapshot notices explain their relationship; browsing newer saved evidence never changes an active review's frozen baseline.

Inspect cited source opens the attached frozen artifact. Find supporting evidence searches the frozen snapshot for equivalent evidence; it is bounded and not exhaustive. Discuss helps you understand, Challenge tests an interpretation, and Help me resolve this coaches next work. An absent source must not be replaced by a guessed file link.

Only authoritative corrected or resolved findings are closed. Confirmed, evidence disputed, accepted risk and deferred findings remain open. Inspecting, searching, recommending and completing a review do not close a finding; work after the snapshot cannot close it on that earlier snapshot.

Button-triggered instructions are recorded as review actions, not as words the student wrote. Unsent drafts are preserved and remain private. Optional review context and other concerns stay secondary to the current concern and conversation.

In Help, artifact, challenge and exit dialogs, Tab and Shift+Tab stay within the dialog, Escape dismisses through the existing close action, and focus returns to a usable opener. Escape does not finish a review. Closing a pending challenge does not cancel its submitted request.

Verify `/health` version 0.18.0, `reasoning_validation_mode` shadow, `review_planning_mode` shadow, and `/ready` `database_connected/migration_current`. Record the actual merged SHA, successful workflow URL and image/revision before publication.

12. Routine administration and semester posture

- Use Studio Semester Setup for course term, sections, roster, teams, review calendar, staff, activation, and archive.
- Treat setup → active → archived as authoritative lifecycle. Setup is administrative preparation; Active is normal student operation; Archived is historical/read-only and cannot grant current student/team/review authority.
- Keep one warm Container App replica during normal semester operation.
- Use `./scripts/azure/etis-azure` status for quick checks and acceptance after deployments.
- Use repository reset only for deliberate re-onboarding; never force-bind arbitrary `Team.repo_full_name` state in the database.
- Review budget/cost signals as stewardship telemetry, not grades or automated shutdown triggers.

13. Upgrade and rollback

13.1 Normal upgrade

1. Branch from current main.
2. Implement the smallest controlled change.
3. Run targeted tests, full pytest, compilation/checks, one-Alembic-head validation where applicable, and diff review.
4. Push/open PR and require green CI.
5. Merge to main.
6. Run Deploy Azure for the merged commit.
7. Run `etis-azure` acceptance.
8. Record material acceptance findings; avoid speculative changes to a healthy release.

13.2 Application rollback

Container Apps runs Single revision mode. Rollback is a controlled redeployment of a known-good immutable ACR image tagged by Git commit SHA, not a traffic switch to an indefinitely retained active revision. Do not rebuild an old commit merely to recreate an image if the immutable known-good image already exists.

13.3 Database changes

Schema changes go through Alembic. Never edit the production schema ad hoc. A rollback decision involving an incompatible migration requires explicit recovery planning; application rollback alone does not automatically reverse database schema changes.

14. Troubleshooting

Symptom	First checks	Likely boundary / action
Can sign in, cannot enter Studio	Confirm active roster/section or staff assignment.	Authentication succeeded; course authorization is missing/stale.
Entra callback error	Compare ETIS_WEB_ORIGIN and exact Entra redirect URI.	Identity configuration/callback alignment.
GitHub identity wrong	Use relink/account selection; check immutable GitHub ID collision.	OAuth identity boundary; do not change repository binding as a shortcut.
Repository verification fails	Confirm App is installed for exact repo and Only select repositories; confirm owner/admin authorization.	GitHub App trust boundary.
GitHub App install returns but Step 2 pending	Return to My Team and explicitly verify repository access.	Expected two-step trust flow.
/health OK, /ready not ready	Check PostgreSQL connectivity and Alembic revision/head.	Database or migration readiness.
First request slow	Check minReplicas via etis-azure config/drift/replicas.	Scale-to-zero drift or replica issue.
Deployment green but wrong live behavior	Run etis-azure drift and smoke; verify latest revision/traffic and source baseline.	Runtime drift or external-path issue.
Cost command WARN 429	Wait; do not hammer Cost Management.	Advisory telemetry throttling, not app health.
Secret/provider failure	Confirm secret name/reference, managed identity access, and environment configuration; never print secret values.	Key Vault/provider configuration.

15. Decommissioning

ARCHIVE IS NOT DELETION

Normal semester archive preserves engineering records and ends normal student operation. Infrastructure deletion is a separate governance action after retention requirements are satisfied.

1. Archive the term through the authoritative lifecycle and verify student operational access ends.
2. Confirm retention requirements for frozen evidence, review turns, findings/dispositions, memberships, and AI usage records.
3. Take/validate the required final database backup/PITR checkpoint and record the restore window.

4. Export required operational/audit evidence.
5. Disable general access/callbacks before destructive infrastructure actions.
6. Revoke/delete GitHub App installations and rotate/revoke private key if retiring the application.
7. Remove/rotate GitHub OAuth and Entra secrets; remove callbacks if the service will no longer be hosted.
8. Delete runtime, database, registry, vault, monitoring, networking, and supporting Azure resources only after retention authorization.
9. Remove DNS/certificate after intentional service shutdown.
10. Close budgets/alerts and record decommission date and retained records.

Appendix A. Greenfield installation checklist

- ☐ Administrative accounts/DNS authority ready
- ☐ Repository cloned; local pytest green
- ☐ Entra app/callback/client secret configured
- ☐ GitHub OAuth identity App/callback configured
- ☐ GitHub App public; Setup URL/Redirect on update configured
- ☐ GitHub App selected-repository read permissions verified
- ☐ Production GitHub environment restricted to main
- ☐ All required secrets and variables configured
- ☐ Azure deployment workflow completes successfully
- ☐ DNS/TLS/callback set aligned
- ☐ etis-azure acceptance PASS
- ☐ Normal Loyola MFA path verified
- ☐ Personal and organization GitHub onboarding verified
- ☐ minReplicas 1 / maxReplicas 5 confirmed
- ☐ Monitoring/alerts/PITR/budget verified
- ☐ Semester data prepared before activation

Appendix B. Required variables and secrets reference

Use Section 7 as the normative classification. Names may be present in documentation; secret values may not. When moving ETIS to another institution, keep the variable names stable unless intentionally changing the deployment contract, and replace institution-specific values such as domain, tenant, bootstrap owner, production-test identity, section/team keys, and alert recipient.

Appendix C. Acceptance and recovery command reference

```
./scripts/azure/etis-azure help
./scripts/azure/etis-azure doctor
./scripts/azure/etis-azure status
./scripts/azure/etis-azure config
./scripts/azure/etis-azure health
./scripts/azure/etis-azure replicas
./scripts/azure/etis-azure revisions
./scripts/azure/etis-azure logs
./scripts/azure/etis-azure smoke
./scripts/azure/etis-azure drift
./scripts/azure/etis-azure budget
./scripts/azure/etis-azure cost
./scripts/azure/etis-azure acceptance
```

MUTATING SCALE OVERRIDE

`./scripts/azure/etis-azure scale <min> <max>` changes live Azure runtime configuration and requires deliberate confirmation. Bicep remains authoritative. Use this only as a guarded operational override, not as a replacement for source-controlled infrastructure changes.

Local UI simulator and deployment guardrails — v0.18.1 candidate

Local simulator

For a synthetic student walkthrough, from the repository root run `python3 -u tools/student_ui_simulator.py` and open `http://127.0.0.1:8766/`. The separate simulator requires no GitHub link, Azure database, or paid model and must not be represented as production data.

Simulation-only scenarios

Use the simulator to seed an older selected review with newer saved Evidence Room evidence, mixed open/finished sessions, long transcripts and failed replies. This validates UI behavior but not real GitHub/Entra/model integration.

Production release procedure

Merge only after branch review and green CI. Dispatch `.github/workflows/deploy-azure.yml` from the accepted merged commit, explicitly selecting `reasoning_validation_mode=shadow` and `review_planning_mode=shadow` because workflow defaults are legacy. Confirm head SHA, `/health`, `/ready`, revision and authenticated student smoke. Do not silently change evaluation modes.

Release boundary

The local simulator lives under `tools/` and is not packaged by the production API Dockerfile. Normal production student APIs and evaluation authority remain unchanged.